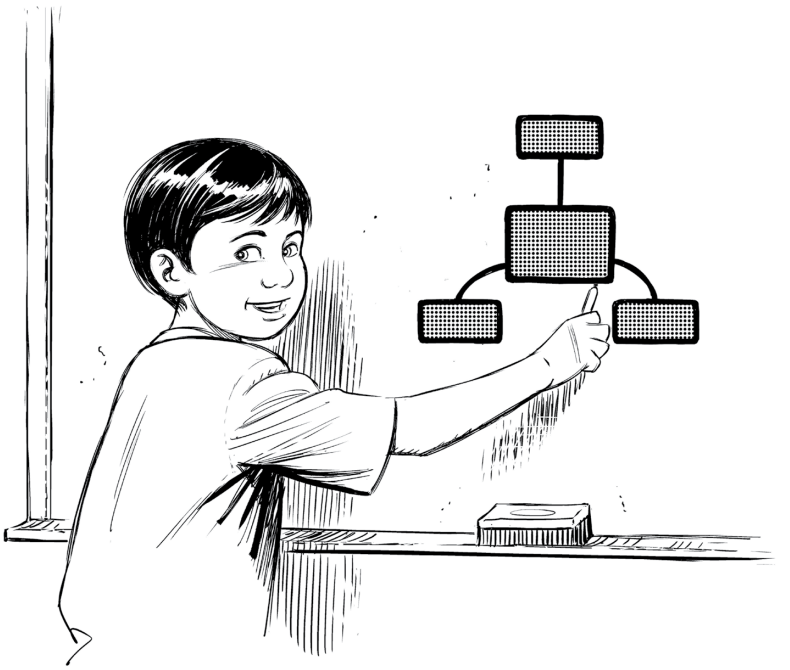
2 architectural characteristics
Know Your Capabilities



What does your architecture need to support? Architectural characteristics (the capabilities of an architecture) are the fundamental building blocks of any system. Without them, you cannot make architectural decisions, select an architectural style, or in many cases even create a logical architecture. In this chapter you'll learn how to define some of the more common characteristics (like scalability, reliability, and testability), how they influence a software architecture, how they help you make architectural decisions, and how to identify which ones are important for your particular situation. Ready to add some capabilities to your software architecture?

Causing Lafter

Sillycon Symposia is a startup with a Bay Area feel whose business plan combines technology-themed conferences with comedy. By gathering like minds, Sillycon provides unique offerings for each group and keeps them engaged by keeping them laughing.

How hard could it be to start a social networking site?!

Part of the business plan includes building Lafter, a social media network related to (but not limited to) the conferences Sillycon hosts. The business stakeholders put together a ***requirements document*** for it:

The good news is that they've got the logo figured out. The rest is up to you.



A pretty standard level of detail for a requirements document

Sillycon Symposia is hosting a social media network of like-minded technologists named Lafter.

Users: Hundreds of speakers, thousands of users

Requirements:

Users can register for usernames and approve the privacy policy

Users can add new content on Lafter as a "Joke" (long-form post) or "Pun" (short-form post)

Followers can "HaHa" (indicating strong approval) or "Giggle" (a milder approval message) content they like

Speakers at Sillycon Symposia events have a special icon

Speakers can host forums on the platform related to their content

Users can post messages of up to 281 characters

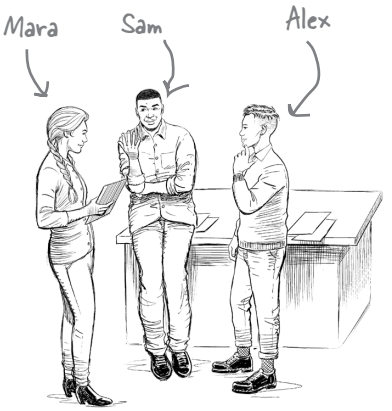
Users can also post links to external content

Additional Context:

International support

Very small support staff

"Bursty" traffic: extremely busy during live conferences

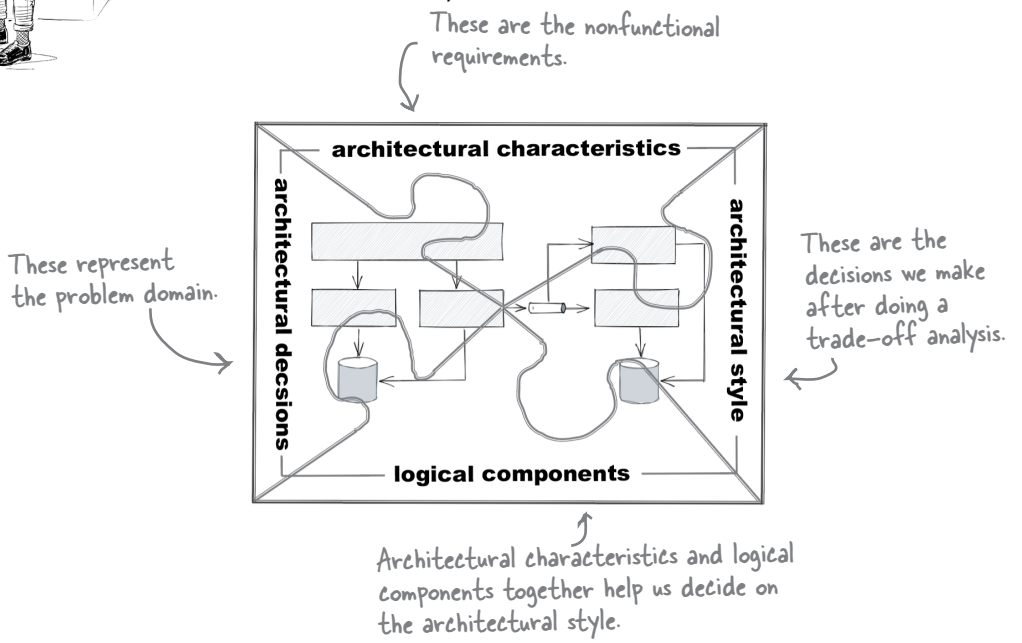


Cubicle conversation

Alex: Look what just landed in my inbox—the Powers That Be want me to be the architect for the Sillycon Symposia social media app, Lafter.

Sam: You have the requirements? You should jump right into the design of the system—it seems really straightforward.

Mara: Well, you can really only do that for the simplest of applications, and I don't think this one qualifies. Remember the diagram I drew on the whiteboard the other day?



You need to analyze both architectural characteristics and logical components before you can choose an architectural style as a starting point.

You can implement just about any application in any architectural style, but some are more suitable than others. Choosing the style before performing this type of analysis is a classic case of putting the cart before the horse.

Sam: Can't we just be super-agile, start with something tiny, and then keep iterating on it until we have the entire system?

Mara: The iterative approach you talk about doesn't quite work like that for architectural characteristics analysis. For example, it's difficult to make a system highly scalable if it wasn't designed for that.

Alex: That makes sense. I guess I need to roll up my sleeves and analyze some architectural characteristics—thanks!

What are architectural characteristics?

You have a problem. You decide, "I'm going to write some software to solve this problem!" The *thing* you're writing software about is called the ***domain***, and designing for it will occupy much of your effort—that is, after all, why you're writing software. However, it's not the only thing an architect must consider—they must also analyze ***architectural characteristics***. Here are a few examples of architectural characteristics that show how different domains have different, but often overlapping, architectural characteristics.

These make up one of the dimensions that help describe your architecture.

Notice how many of them end with "–ility".

Auditability
Banks must provide a way to verify transactions.

Data integrity
Financial transactions must be consistent and accurate.



Bank

Security
Banks require stringent security to protect financial concerns.

Both domains have scalability as an architectural characteristic they must support.

Scalability

Banks must support large numbers of concurrent users.

Online auctions must support a large number of bidders.

Usability
Auction sites must be easy to use for quick and efficient entry of bids.



Online auction

Consistency
Bids must be captured consistently and in order for auctions to work.

Reliability
Auction sites must be reliable—users don't like it if their connection drops in the middle of an auction.

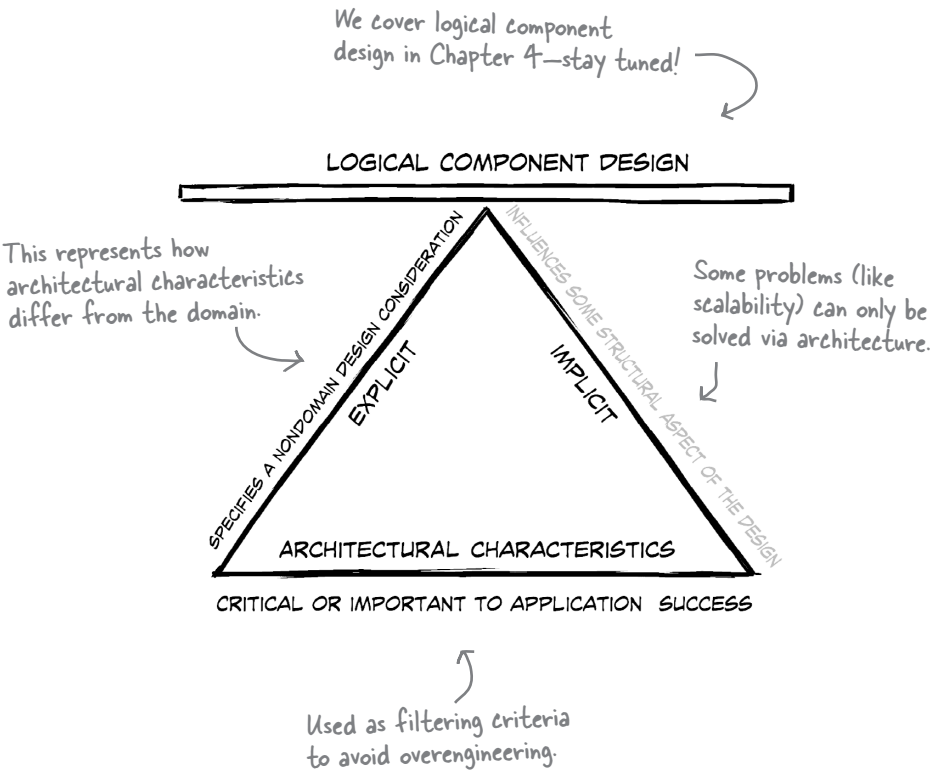
Defining architectural characteristics

Part of your job as an architect is structural design for software systems, for which there are two parts: ***logical components*** and ***architectural characteristics***. Logical components represent the domain of the application—the motivation for writing the software system (we cover these in Chapter 4). If you combine architectural characteristics with logical components, you have the structural considerations for an architecture.

Architectural characteristics are the important parts of the construction process of a software system or application, irrespective of the problem domain. They represent its operational capabilities, internal structure decisions, and other necessary characteristics.

We'll show you lots of examples of architectural characteristics in the upcoming pages, but first we want to cover the concept itself.

We define architectural characteristics in **three parts**, as shown here.

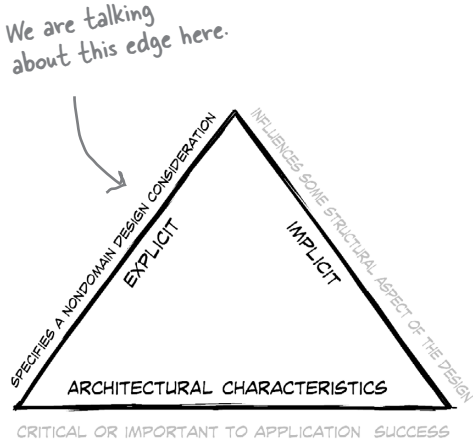


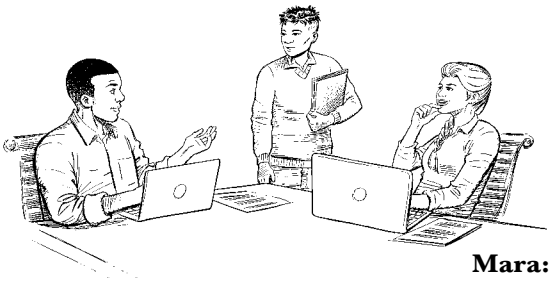
Let's look at each of these edges one at a time.

Characteristics are nondomain design considerations

To define architectural characteristics, we first need to look at what they are ***not***. The *requirements* specify what the application should do; *architectural characteristics* specify operational and design criteria for success, how to implement the requirements, and why certain choices were made. For example, an application's level of performance is an important architectural characteristic that often doesn't appear in requirements documents.

Structural design in architecture can be divided into ***domain and non-domain considerations***. Architectural characteristics represent your design effort to create the capabilities necessary for the project to succeed.





Mara: OK, the business analysts and subject matter experts have toiled away to create both a requirements document and the beginning of a domain design. But we need to work with them to figure out what architectural characteristics we need to support.

Sam: Isn't that part of the domain design? Why does an architect need to get involved at this point?

Alex: Well, our business analysts have never worked on a software porject, so they probably won't understand the impact of one decision versus another.

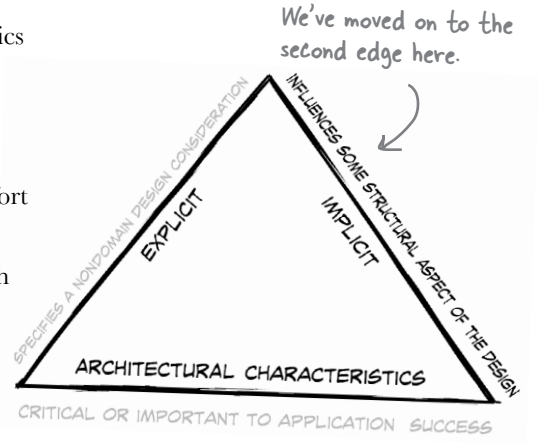
Mara: That's correct—often what seems like a minor difference to a business person makes a big difference for an architect! What they want may turn out to be difficult to support in architecture. That's why it's important for architects to be involved early and often in the design process.

Sam: What kinds of things are we looking for?

Alex: Part of the definition of *architectural characteristics* is "nondomain design considerations." Let's look at what they've designed and see if they've considered things like performance and scalability.

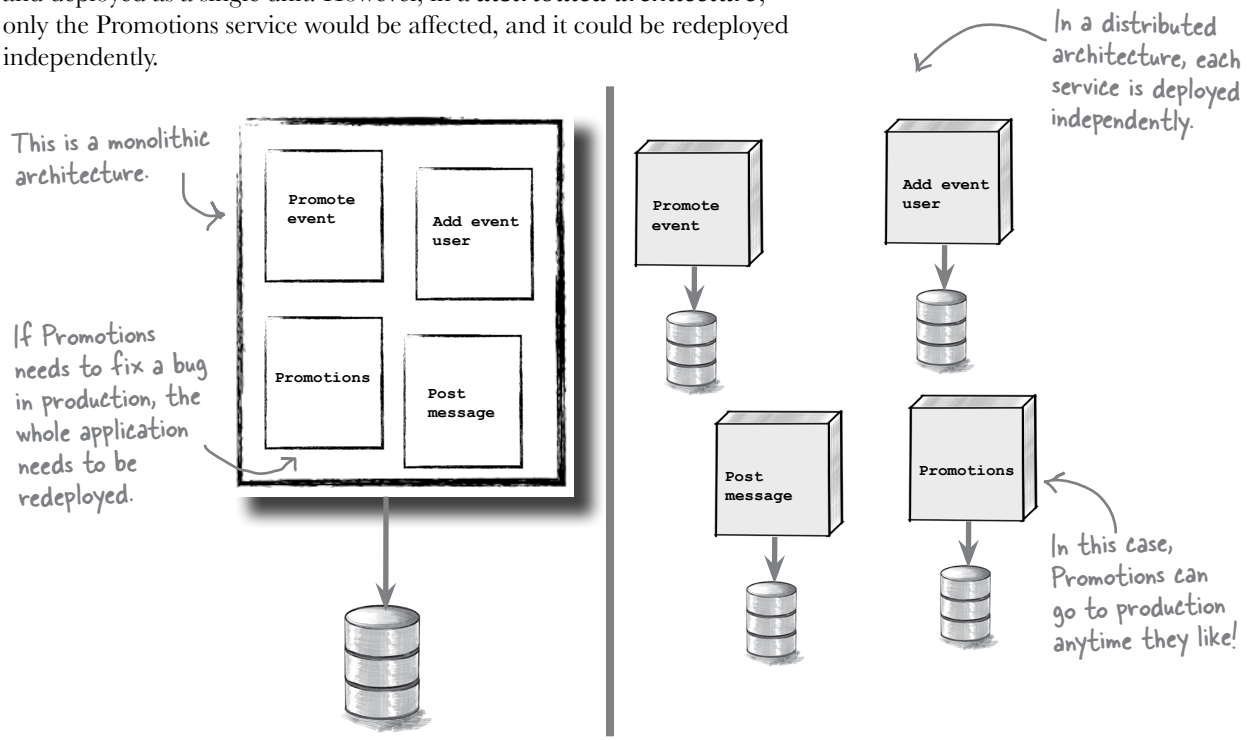
Characteristics influence architectural structure

The primary reason architects try to describe architectural characteristics has to do with architectural considerations: does this characteristic require special structural support to succeed? For example, security is a concern in virtually every project, and all systems must take baseline precautions during design and coding. However, security becomes an *architectural* characteristic when the architect needs to make a special effort to accommodate it.



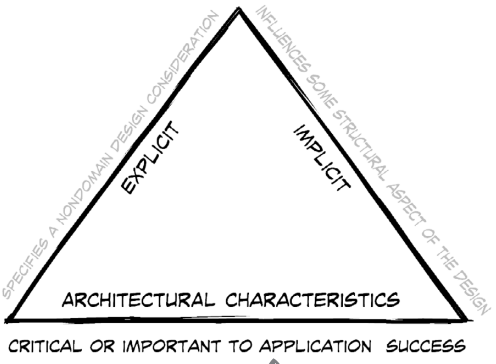
Consider the following potential architecture diagrams for Lafter, which include functionality for marketing upcoming promotions and rules for when each promotion applies. An architect could design this as a ***monolithic architecture***—one with a single deployable unit and matching database—or as a series of independent services.

For the monolithic architecture, the entire application would have to be redeployed when the promotion rules change, because monoliths are built and deployed as a single unit. However, in a ***distributed architecture***, only the Promotions service would be affected, and it could be redeployed independently.



You must consider many trade-offs when making architectural decisions, such as whether to use a monolithic versus distributed physical architecture.

Limit characteristics to prevent overengineering



We are talking about the third edge. Almost there!

Applications *could* support a huge number of architectural characteristics…but they shouldn't. Every architectural characteristic the system must support adds complexity.

The sheer number and variety of architectural characteristics means there are many tempting choices. But as architects, we should try to pick as *few* architectural characteristics as possible, rather than as many as possible. This is because architectural characteristics are:

Impossible to standardize

Different organizations use different terms for the same architectural characteristics. For example, *performance* and *responsiveness* might indicate the same behavior.

It's a good idea to create a "ubiquitous language" (shared vocabulary) for architectural characteristics within your organization—this gives you a fighting chance at creating a usable standard list.

Synergistic

Architectural characteristics affect other architectural characteristics and domain concerns. For example, if you want to make an application more secure, the required changes will almost certainly affect performance negatively (more on-the-fly encryption and other similar changes will lead to performance overhead).

Your takeaway? You often cannot choose one architectural characteristic without considering how it may affect others.

Overabundant

Possible architectural characteristics are extraordinarily abundant, and new ones appear all the time. For example, a few years ago there was no such thing as on-demand elasticity via a cloud provider.

Even the number of categories of architectural characteristics has increased over the last few years, with additions such as cloud constraints and capabilities.

We'll be discussing some categories of architectural characteristics soon.

A common hazard for architects is *overengineering*: supporting too many architectural characteristics and complicating the overall design to little or no benefit. Knowing which architectural characteristics are ***critical or important to application success*** acts as a filtering tool. It help us *eliminate* features that would be nice to have but just end up adding needless complexity to the system.

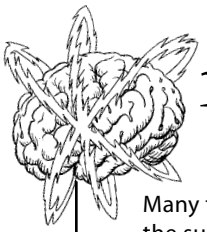
Beware of resume-driven development (RDD)! It's fun to play with new stuff, and we should keep learning, but trying to support too many architectural characteristics in our systems will not align with larger priorities or help the application succeed.



Synergy can be dangerous!

Architects would love to design for architectural characteristics irrespective of the domain design. Unfortunately, the real world refuses to cooperate. When we say that architectural characteristics are ***synergistic***, we mean that changes to one might require changes to other architectural characteristics and/or the domain. No matter how clever you are, no architect can make every single architecture scalable. Some architectures can't scale as successfully as others because of physical constraints such as memory and bandwidth.

Be careful when you change one architectural characteristic; consider how that change may affect other parts of your architecture. The same applies to making changes to domain design, such as component boundaries and distribution—changes to the domain may synergistically affect your architectural characteristics. For example, if you change your application to begin storing users' payment information, the security and data integrity architectural characteristics will also change.



Brain Power

Many things in the the real world are **synergistic**—that is, combining them yields something different than the sum of the parts. See if you can think of some real-world examples of synergy. *Hint: These might include things that are still identifiable (like peanut butter and chocolate) or things that merge (such as emulsions like oil and vinegar).*

Use this space to jot down your ideas.

Consider explicit and implicit capabilities

Some things are *explicit*—stated clearly—whereas others are *implicit*—assumed based on context or other knowledge. Imagine if you saw a bunch of mail and packages piling up outside the door of a home—what conclusions would you draw?

Explicit

Packages are stacked outside a door.

The door is locked.



Implicit

No one is home.

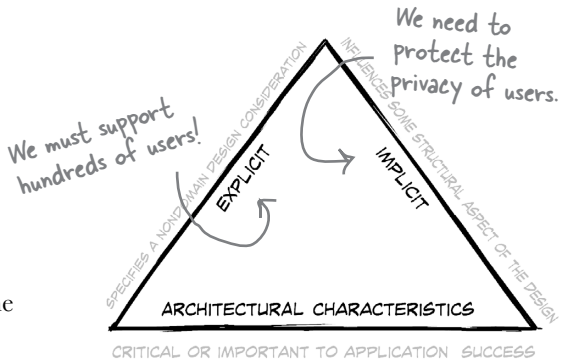
This family orders a bunch of stuff online.

Is this family on vacation?

Explicit architectural characteristics are specified in the requirements for the application.

Implicit architectural characteristics are factors that influence an architect's decisions but aren't explicitly called out in the requirements. Security is often an implicit architectural characteristic: even if it isn't mentioned in the requirements, architects know that we shouldn't design an insecure system.

You must use your knowledge of the problem domain to uncover these architectural characteristics during the analysis phase. For example, a high-frequency trading firm may not specify how critical it is for transactions to complete within milliseconds, but the architects in that problem domain know how important this is.



there are no Dumb Questions

Q: What about important things like good internal structure that no one thinks to ask for?

A: Some implicit architectural characteristics are more subtle, but just as important. For example, architects should pay attention to the application's internal structure as developers create it, to ensure that sloppy coding and other deficiencies don't degrade the longevity of the application. However, virtually no requirements list will specify "Don't mess up the internal modularity of the system as you build it!" or "Make sure the software is maintainable!"

The International Zoo of "-ilities"

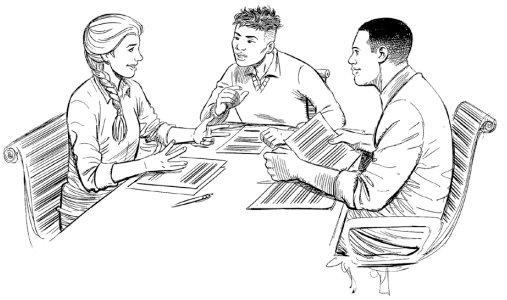
Like the animals in a zoo, architectural characteristics exist along a broad spectrum. Just as animals range from primates to reptiles, architectural characteristics range from low-level code characteristics, such as modularity, to sophisticated operational concerns, such as scalability and elasticity. Unfortunately, there is no "universal list" of architectural characteristics, nor are there any real standards for what many of these terms mean (although people have tried). Instead, each organization interprets these terms for itself.

Additionally, the software ecosystem is constantly adding new concepts, terms, measures, and verifications, providing new opportunities to define *even more* architectural characteristics.

Think this is a big list? Check out https://iso25000.com/index.php/en/iso-25000-standards/iso-25010.

scalability	accessibility	observability
availability	maintainability	testability
interoperability	reusability	portability
security	simplicity	feasibility
stability	reliability	usability
agility	integrity	performance
traceability	localizability	auditability

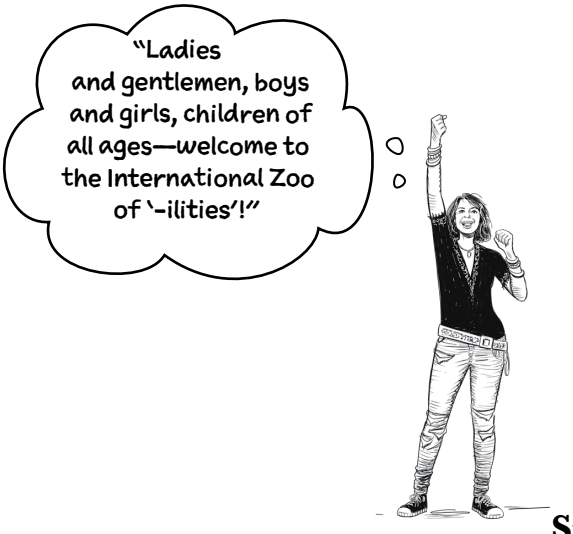
This is not the complete list. There is no complete list!



Sam: We're supposed to define architectural characteristics for Lafter, but I can't seem to find a standard list anywhere.

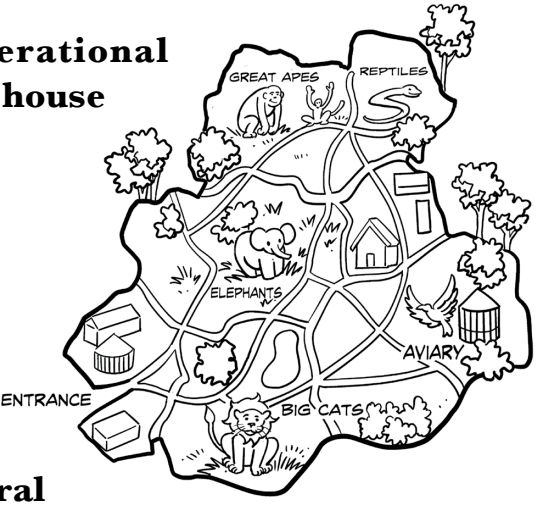
Alex: Gosh, there are so many possibilities...

Mara: That's why I like to categorize them. Remember the old zoo maps that broke the zoo into "houses" and "enclosures" for each type of animal? That same kind of categorization can work here. It's sort of like the genus and species of architectural characteristics.



Process enclosure

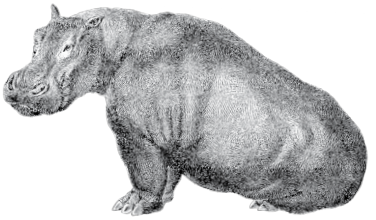
Operational house



Structural pen

Cross-cutting savannah

Process architectural characteristics



Process architectural characteristics are where the software development process intersects with software architecture. They reflect the decisions about the mechanics of building software.



modularity

The degree to which the software is composed of discrete components. Modularity affects how architects partition behavior and organize logical building blocks.



testability

How complete the system's testing is and how easy these tests are to run, including unit, functional, user acceptance, and exploratory tests.

"Testability" refers to testing at development time (such as unit testing), rather than formal quality assurance.



agility

A composite architectural characteristic that encompasses testability, deployability, modularity, and a host of other architectural characteristics that facilitate and enable agile software development practices.

Agility is a composite architectural characteristic we'll discuss later in this chapter—stay tuned!

Look out for this on the next page. Many architectural characteristics cut across categories, as you'll see in the next few pages.



deployability

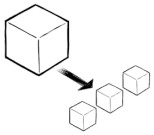
How easy and efficient it is to deploy the software system.

This is one of the many architectural characteristics that make up "agility."



extensibility

How easy it is for developers to extend the system. This may encompass architectural structure, engineering practices, internal design, and governance.



decouple-ability

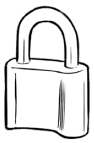
Coupling describes how parts of the system are joined together. Some architectures define how to *decouple* parts in specific ways to achieve certain benefits; this architectural characteristic measures the extent to which this is possible in a software system.

Yes, we know this is a made-up word. That happens a lot in software architecture!

Structural architectural characteristics

Structural architectural characteristics affect the internal structure of the software system, including factors like the degree of coupling between components and the relationships between different integration points.

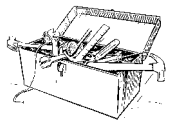




security

How secure the system is, holistically. Does the data need to be encrypted in the database? How about for network communication between internal systems? What type of authentication needs to be in place for remote user access?

Security appears in every application, as an implicit or explicit architectural characteristic.



maintainability

How easy it is for architects and developers to apply changes to enhance the system and/or fix bugs.

Portability can apply to any part of the system, including the user interface and implementation platform.



portability

How easy it is to run the system on more than one platform (for example, Windows and macOS).

This is one of those characteristics that belong to more than one category.



extensibility

How easy it is for developers to extend the system. This may encompass architectural structure, engineering practices, internal design, and governance.

Some architectural characteristics cover development concerns rather than purely domain concerns.



localization

How well the system supports multiple languages, units of measurement, currencies, and other factors that allow it to be used globally.

Another flavor of localization is internationalization (i18n).

Operational architectural characteristics

Operational architectural characteristics represent how architectural decisions influence what operational team members can do.





availability

What percentage of the time the system needs to be available and, if 24/7, how easy it is to get the system up and running quickly after a failure.

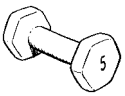
Usually represented as a number of "nines" (99.999% uptime = 5 nines, a bit under 6 minutes/year).



recoverability

How quickly the system can get online again and maintain business continuity in case of a disaster. This will affect the backup strategy and requirements for duplicated hardware.

A good example of the axiom that you can take any adjective and add "–ility" to make a new architectural characteristic!



robustness

The system's ability to handle errors and boundary conditions while running, such as if the power, internet connection, or hardware fails.



performance

How well the system achieves its timing requirements using the available resources.

As you will see shortly, "performance" has many different aspects.

When these are important, they are very important.



reliability/safety

Whether the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it endanger people's lives or cost the company large sums of money? Common for medical systems, hospital software, and airplane applications.

Some "–ilities" are easier to achieve than others. This one is often difficult.



scalability

How well the system performs and operates as the number of users or requests increases.

Our Lafter application will definitely need this!

Cross-cutting architectural characteristics



As much as we'd like a nice, orderly zoo of architectural characteristics, platypuses still show up! Lots of important characteristics defy categorization.



security

How secure the system is, holistically. Does the data need to be encrypted in the database? How about for network communication between internal systems? What type of authentication needs to be in place for remote user access?

This is one of those architectural characteristics that are always present. It also happens to be a cross-cutting concern.

Authentication and authorization are aspects of security.



legal

How well the system complies with local laws about data protection and about how the application should be built or deployed.



authentication/authorization

How well the system ensures users are who they say they are and makes sure they can access only certain functions within the application (by use case, subsystem, web page, business rule, field level, etc.).



privacy

How well the system hides and encrypts transactions so that internal employees like data operators, architects, and developers cannot see them.

Many countries and regions have strict laws governing privacy, making consistency for international applications tricky.

Many government agencies around the world require a baseline level of accessibility.



accessibility

How easy is it for all your users to access the system, including those with disabilities, like colorblindness or hearing loss.



usability

How easy is it for users to achieve their goals. Is training required? Usability requirements need to be treated as seriously as any other architectural issue.

This is a great example of how ambiguous architectural characteristics can be: "usability" can also refer to user experience design.

Who Does What?

So many architectural characteristics! We had a nice database that listed the ones that are most important for Lafter, along with their definitions, but somehow the index became corrupted and we lost the linkage. Can you help restore them by drawing a line from each architectural characteristic to its definition?

scalability	Describes how well the components in the system create well-defined groupings and boundaries between components.
deployability	The system's ability to recover from problems such as a power, internet connection, or hardware failure.
modularity	How easy is it for all users to access the system, including those with disabilities like colorblindness or hearing loss.
robustness	How easy it is for architects and developers to apply changes to enhance the system and/or fix bugs.
accessibility	Describes how well the system handles a large number of concurrent users while maintaining reasonable performance.
maintainability	Describes the cadence, efficiency, and reproducibility of deployments.

Solution on page 74

there are no Dumb Questions

Q: Where can I find a standard list of architectural characteristics?

A: No standard list really exists (despite several futile efforts) because the software development ecosystem constantly shifts and changes. Anyone trying to create a standard list is trying to hit a moving target.

Q: Isn't security required for every application?

A: It depends! While it's a common concern, if you design a free intra-office lunch-ordering system, the only security concern lies with others finding out that you order an egg salad sandwich every day.

Q: Doesn't every application require availabilty?

A: You guessed it—it depends! Again, availability is a common concern for most applications, but if the mythical sandwich-ordering system mentioned above fails, the only real downside is that everyone has to get their own lunch.

Q: Can I choose any combination of architectural characteristics for my application?

A: Some architectural characteristics oppose one another. For example, architects find it challenging to design for both high performance and scalability. Determining the most important architectural characteristics for a system is only part of the design process. Combining them with logical component design will point you to an appropriate architectural style.

Q: What does it mean if you don't choose an architectural characteristic like availability in your requirements?

A: The architectural characteristics you choose provide a guideline for the appropriate architectural style. If an architect doesn't choose availability, it doesn't mean they will purposefully design the system to have poor availability. Rather, it's an indication of priority: trading off one architectural characteristic for another.

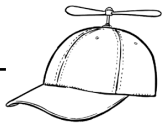


Exercise

Welcome to "Take It or Leave It!" The rules of this game are simple—we'll give you a business requirement that might come up for the Lafter application, and two architectural characteristics. As you know, everything in architecture is a trade-off, so if you attempt to optimize one, you probably won't do as well with the other. Your job is to tell us which characteristic you rate as a higher priority for that requirement. You'll find our thoughts at the end of the chapter.

"We need to get this to market ASAP!"	fault tolerance	agility
"Money's tight, folks!"	scalability	simplicity
"Oh, wow, this conference is going to be our biggest yet."	high availability	maintainability
"We want to start storing users' credit card information."	security	recoverability
"This site is going to be very popular upon launch."	agility	elasticity

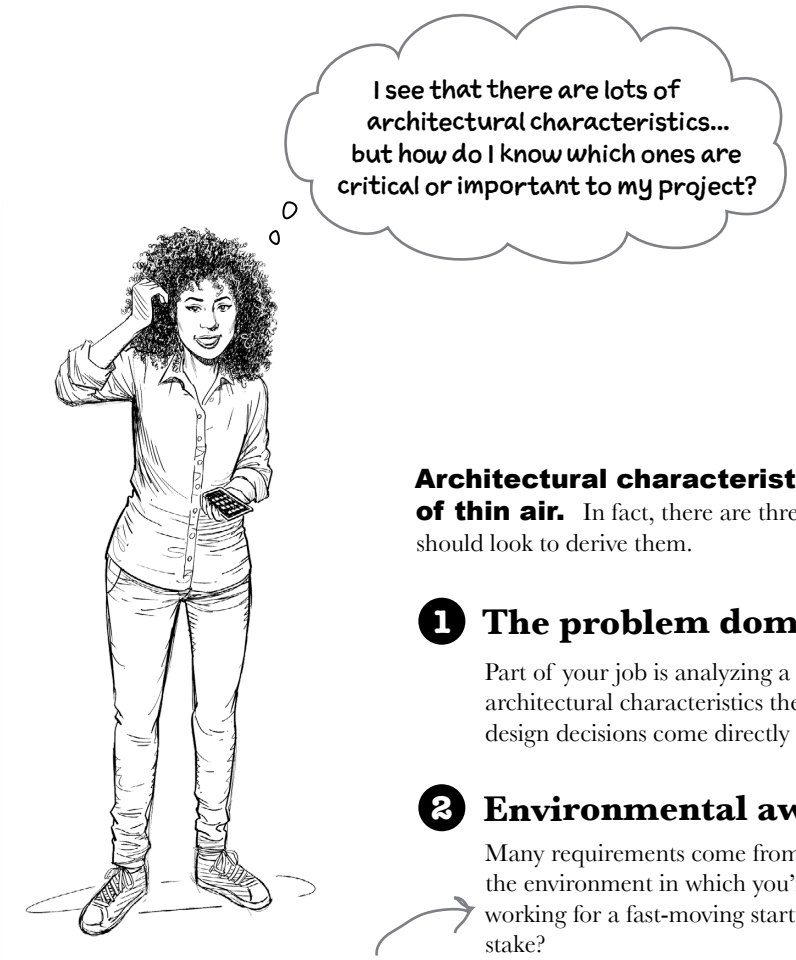
Solution on page 75



Geek Note

The software architecture world lacks a standard term for what we call *architectural characteristics*. Here are some of the terms people often use, and why we don't care for them.

Most teams still call them *nonfunctional requirements*, which is misleading because architectural characteristics are indeed functional—they just don't concern the domain. Calling them nonfunctional downplays their importance. Other teams call them *system quality attributes,* which implies an activity that happens at the end of the project rather than the beginning. Another common name is *cross-cutting requirements,* which is the one we dislike the least—but it contains the word *requirement*, which entangles it with domain behaviors, which come from requirements, as opposed to *capabilities*, which are defined by architectural characteristics.



Architectural characteristics don't just appear out of thin air. In fact, there are three different sources from which you should look to derive them.

1 The problem domain

Part of your job is analyzing a problem to determine what architectural characteristics the system requires. Many structural design decisions come directly from the problem domain.

2 Environmental awareness

Many requirements come from having a good understanding of the environment in which you're operating. For example, are you working for a fast-moving startup, or a large enterprise with a lot at stake?

3 Holistic domain knowledge

Sure, you're working with a particular problem domain—but we can assure you that the domain is a lot bigger than your particular focus. Let's say you're building out a payment system. While understanding what's required of you is important, you'll reveal architectural characteristics if you understand the financial world, finance industry regulations, and customers' habits.

Let's look at each of these sources in turn.

Recall that some architectural characteristics are "implicit." Many implicit characteristics emerge from these two sources.

Sourcing architectural characteristics from the problem domain

Architects derive many of the necessary architectural characteristics from the problem domain—it is, after all, the motivation for writing the software in the first place. That means you must *translate* the items stated in requirements documents into their corresponding architectural characteristics. For example, the Lafter requirements specify "thousands of users." As an architect, you must dig deeper to more accurately determine how many users are expected (scalability), how many of them will be there at the same time (concurrency), and how rapidly they'll show up (elasticity).



Exercise

Domain requirements are often a rich source of architectural characteristics. For example, our Lafter application needs to support large numbers of users, so scalability will be one necessary characteristic. Can you uncover more? Here are the requirements again:

Users: hundreds of speakers, thousands of users

Requirements:

- Users can register for usernames and approve the privacy policy
- Users can add new content on Lafter as a "Joke" (long-form post) or "Pun" (short-form post)
- Followers can "HaHa" (indicating strong approval) or "Giggle" (a milder approval message) content they like
- Speakers at Sillycon Symposia events have a special icon
- Speakers can host forums on the platform related to their content
- Users can post messages of up to 281 characters
- Users can also post links to external content

Additional Context:

- International support
- Very small support staff
- "Bursty" traffic: extremely busy during live conferences

Architectural characteristics

Scalability

→ Solution on page 76

Sourcing architectural characteristics from environmental awareness

You know a lot about where you work (maybe too much, in some cases!), and that will naturally drive your architectural characteristics analysis. For example, an architect working for a fast-moving startup will prioritize agility whether it is specified or not.

Sorry, but this means you're going to have to start paying attention in those business prioritization meetings!

It is important to understand ***organizational priorities*** so we can make more durable decisions. For example, let's say we must decide how to integrate two subsystems. The choices are a customized but highly suited protocol or an industry-standard protocol that will require a little more effort. In a vacuum, we might choose the first. However, if we know that the organization's goal is to engage heavily in mergers with other companies, that fact could tip our decision toward the more open solution.

Architects can't make decisions in a vacuum—context is always important.

Some architects stay within particular domains exactly because they have the advantage of domain knowledge.

Sourcing architectural characteristics from holistic domain knowledge

You have also no doubt absorbed a lot of ***domain knowledge***: information that isn't explicitly spelled out in the requirements but that you implicitly understand about important aspects of the domain.

Suppose Lafter has decided to run a promotion at a local university to entice students to sign up (they go to a lot of conferences, and some of them have a sense of humor). We need to design an application that handles sign-ups for the promotion day. To make the math easy, assume that the school has 1,000 students and they have 10 hours to sign up. Should we design the system using a consistent scale, implicitly assuming that the students will distribute themselves evenly during the sign-up process? (Have we *met* any university students?)

Never underestimate some university students' ability to procrastinate.

Based on real-world experience, we can guess that this won't work. Think about what you know about the target demographic. Some students are hyperdiligent; some tend to procrastinate. Thus, the actual design must handle an elastic burst of students in the first hour (as the Type A individuals rush to get in line), stay mostly idle for the bulk of the day, and then handle another elastic burst just before the sign-up window closes, to accommodate all the stragglers.

One of the most dangerous discoveries in life is how much you can procrastinate and still (mostly) get the job done.

Architects must make use of all available information sources to understand the full range of trade-offs inherent in our architectural decisions.



Solutions versus requirements

Customers often come to architects with *solutions* rather than requirements. For example, back in the 1970s, the US Air Force commissioned a fighter jet and included a requirement that it be capable of achieving speeds up to Mach 2.5. The designers tried, but the technology of the time just wasn't sufficient to meet the requirement. They went back to the Air Force and asked: "Why does it need to go Mach 2.5?" The answer was, "Well, these things are expensive, so we want it to be able to flee a fight if necessary." With that knowledge in mind, they went back and designed the F-16 fighter jet. It had a maximum speed of Mach 2.1, but it was the most maneuverable and fastest-accelerating jet ever created.

When users bring us solutions rather than requirements, it's architects' job to imitate an annoying toddler and keep asking "But why?!?" enough times to uncover the *actual* requirements hidden within the solutions.



Exercise

It's sometimes difficult to distinguish requirements from solutions. Here are some responses you might get when you ask "why?" that could indicate something might be one or the other. Can you identify which indicate requirements and which solutions?

"We need a system to track user preferences and customizations, then save them between sessions."

☐ **requirement** ☐ **solution**

"Do we really need to build our own survey service? Surely we can find one that does what we need."

☐ **requirement** ☐ **solution**

"An enterprise service bus would solve some of our current problems (albeit with some changes and work-arounds) and it offers extreme extensibility."

☐ **requirement** ☐ **solution**

"According to the friendly sales rep, this software package does all the things accounting needs, now and in the near future."

☐ **requirement** ☐ **solution**

→ Solution on page 77

Composite architectural characteristics

Alex: The business analyst asked if we can make sure the system is "reliable." What do they mean?



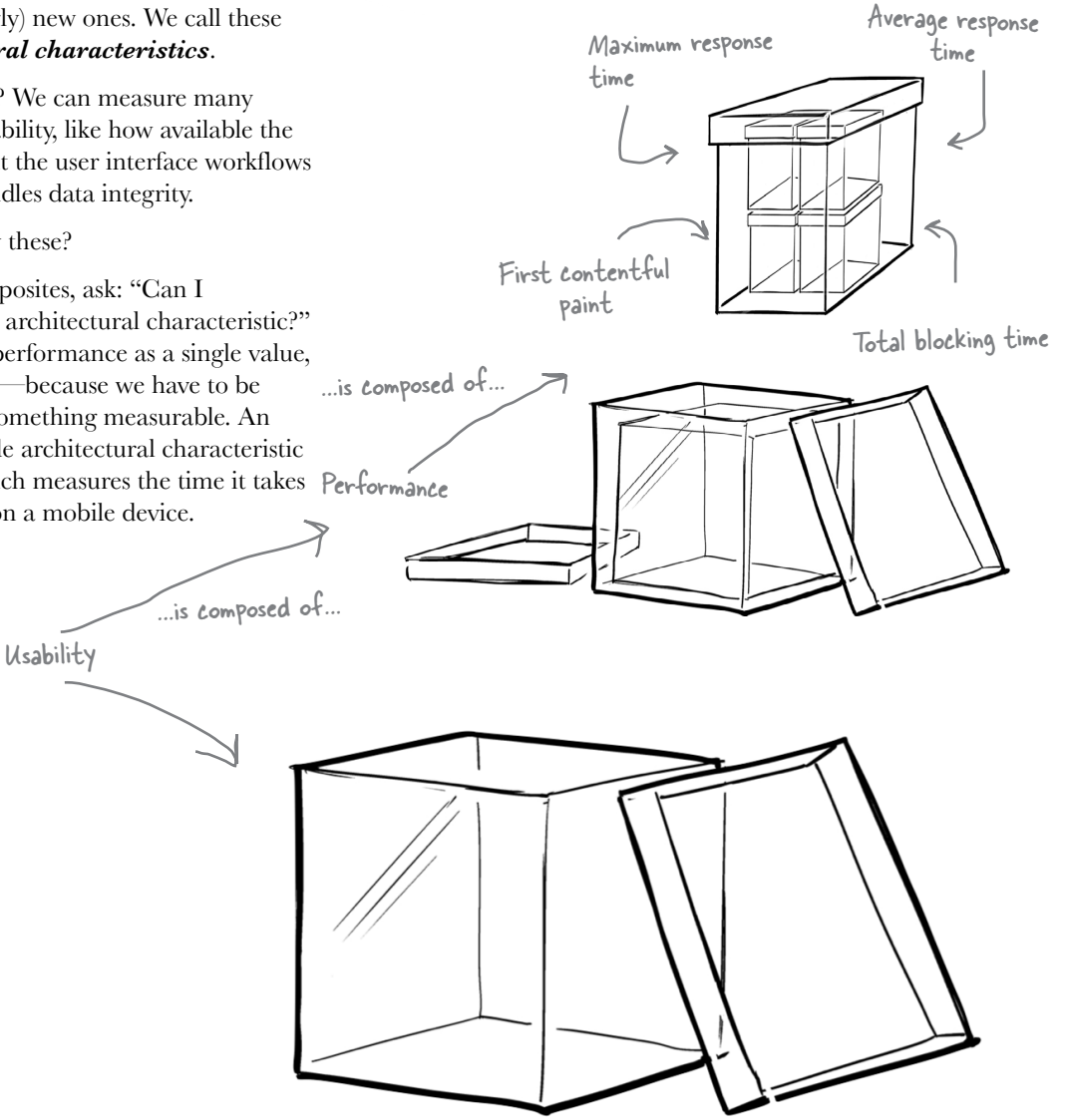
Sam: Wow, I can think of a lot of ways to define "reliable" for a piece of software.

Mara: This happens a lot. A ***composite*** is a combination of two or more things, and often architectural characteristics combine with each other to create (seemingly) new ones. We call these ***composite architectural characteristics***.

What does *reliable* mean? We can measure many different aspects of reliability, like how available the system is, how consistent the user interface workflows are, and how well it handles data integrity.

Alex: How do I identify these?

Mara: To identify composites, ask: "Can I objectively measure this architectural characteristic?" While we often discuss performance as a single value, it's actually a composite—because we have to be more specific to get to something measurable. An example of a measurable architectural characteristic is *first contentful paint*, which measures the time it takes for a web page to load on a mobile device.



Priorities are contextual

It's impossible to choose the same set of architectural characteristics for every project. The set of architectural characteristics you choose for a particular application, and how you prioritize each one, will differ based on context.

Sharpen your pencil

Context matters. At the top, we've listed several architectural characteristics. Below that are three application scenarios. For each scenario, rank each characteristic based on how important it is for that type of application. *Hint: Some applications won't need all of them.*

Scalability

Performance

Extensibility

Security

Data integrity

Scenario #1

An ecommerce site in a competitive market

1 _____

2 _____

3 _____

4 _____

5 _____

Scenario #2

A system for an enterprise whose goal is to grow via mergers

1 _____

2 _____

3 _____

4 _____

5 _____

Scenario #3

An application to automate standardized testing and grading for university admissions

1 _____

2 _____

3 _____

4 _____

5 _____

Solution on page 78



Congratulations, you have yet *another* job.

You're right to be skeptical about how sophisticated an understanding your coworkers have of architectural concepts. That means you have one more job as an architect: translation!

As much as it would be nice for our colleagues to learn our language, architects are generally the ones who have to translate the business's goals into *identifiable and measurable* architectural characteristics.

Lost in translation

It's not unusual for business experts and analysts to state (or subtly suggest) a requirement without realizing it, hidden in plain English. It's your job, as a software architect, to read between the lines, find the requirements, and translate them into architectural characteristics. Here are a few examples.

When business analysts and subject matter experts say:	Software architects translate:
"Lafter is constantly changing to meet new marketplace demands."	• Agility • Modularity • Extensibility
"Due to new regulatory requirements, we must complete end-of-day processing on time."	• Performance • Recoverability • Scalability
"Our plan is to engage heavily in mergers and acquisitions in the next three years."	• Résumé-ability • Integratability • Interoperability
"We have a very tight time frame and a fixed scope and budget for this project."	• Feasibility • Simplicity • Affordability

Good modularity allows for faster change without rippling side effects.

We must perform well but also recover quickly in case of error.

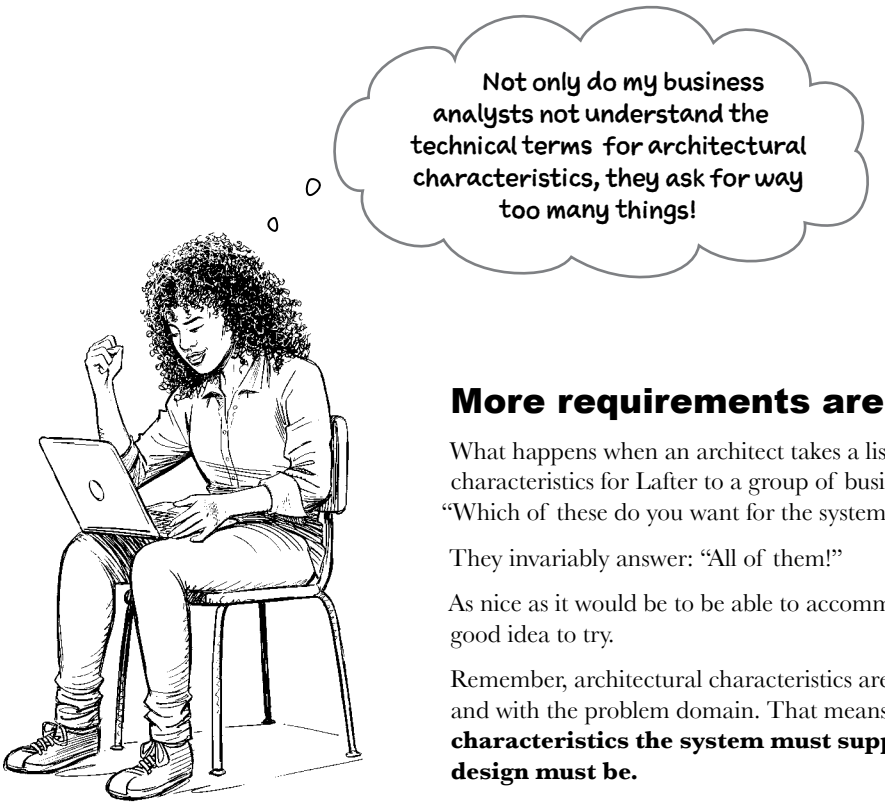
"The ability to update your résumé." Many people would rather not work in a place undergoing constant mergers.

More of an architect-the-person characteristic.

Of course, no one would ever ask for this impossible combination... ahem.

Feasibility—evaluating whether something is possible—is an underutilized architecture "-ility."

Architects often have a unique perspective of what's possible within a given time frame.



More requirements are NOT better.

What happens when an architect takes a list of possible architectural characteristics for Lafter to a group of business users and asks them, "Which of these do you want for the system?"

They invariably answer: "All of them!"

As nice as it would be to be able to accommodate that request, it's not a good idea to try.

Remember, architectural characteristics are ***synergistic*** with each other and with the problem domain. That means **the more architectural characteristics the system must support, the more complex its design must be.**

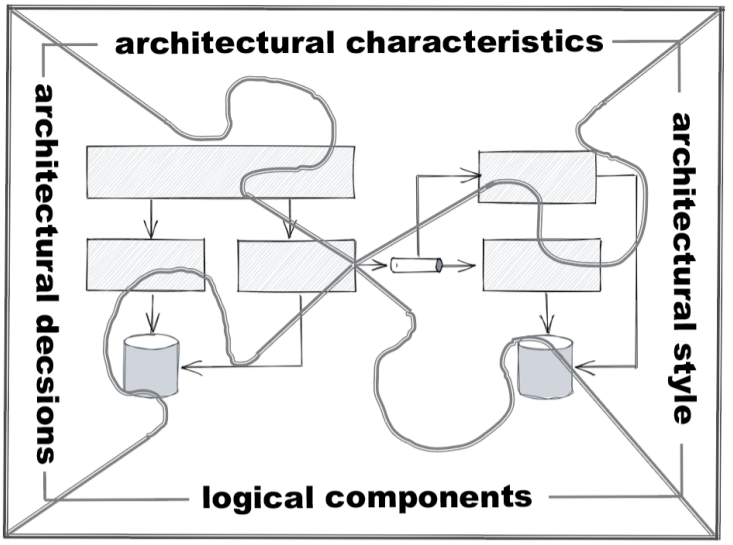
When undertaking structural design for a system, architects must find a balance between domain priorities and the architectural characteristics necessary for success.

Architectural characteristics and logical components

Before we tell you how to go about trying to balance architectural characteristics with each other and the domain, we want to show you how architectural characteristics and logical components are two sides of the same coin. You see, both of them aim to support the problem domain (also known as the reason you are writing software to begin with).

Architectural characteristics = capabilities

Architectural characteristics describe the *kinds of capabilities* your solution will support, rather than the *behavior* of the application, which is based on requirements.



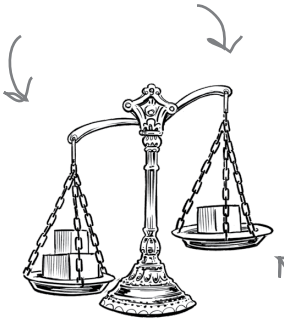
Logical components = behavior

Logical components, covered in depth in Chapter 4, represent the design of the system you are attempting to implement in software in order to solve the fundamental problem at hand.

Balancing domain considerations and architectural characteristics

Architects use architectural characteristics and logical component analysis to determine the appropriate architectural style, and the Lafter application is no exception. You need to strike a balance between the two.

Imagine the left side to be domain considerations and the right side to be architectural characteristics.



No architectural characteristics

Sometimes we don't take the time to analyze architectural characteristics before designing the system, leading to expensive and time-consuming rework as we discover that our system fails to exhibit the necessary architectural characteristics.

Projects that ignore needed architectural characteristics can deliver failure faster.

Remember, architectural characteristics are synergistic with domain and other architectural characteristics.

Good balance between...

In this scenario, we have achieved a balance in our design decisions between architectural characteristics and domain considerations.



...architectural characteristics and domain considerations

This allows us to achieve operational and structural goals without overengineering.

Too many architectural characteristics

Unfortunately, architects sometimes retreat to an ivory tower and spend too much time analyzing architectural characteristics, or identifying too many of them to be useful. This leads to overengineering, wasting time and effort that could be spent on implementation and ongoing maintenance.



Many systems that try to support too many architectural characteristics end up with too little space left to support the domain.

Limiting architectural characteristics

When the business stakeholders want *all* of the possible architectural characteristics, how can you limit their enthusiasm?

The magical number 7

One useful guideline for the conversation between architects and business analysts is to **limit the number of architectural characteristics they can choose to seven**. Why seven? Psychological research indicates that people remember items in chunks of seven (one of the reasons that early phone numbers were seven digits). It's also large enough to provide some variety without creating a paradox of choice by offering too many.

"The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information" is a famous paper from 1956 by psychologist George Miller.

We created a worksheet to help architects work with other stakeholders to arrive at a reasonable number. This is a demo; you'll get to use it on the next page.

Once you've settled on the most important driving characteristics, you get to pick three. Just put 1,2, and 3 next to the ones that matter the most to you.

Driving characteristics are architectural characteristics that drive important design decisions.

Implicit characteristics become driving characteristics if they influence structural decisions.

Top 3	Driving Characteristics	Implicit Characteristics
☐		*feasibility (cost/time)*
☐		*security*
☐		*maintainability*
☐		*observability*
☐		
☐		
☐		

We've found these four to be pretty common implicit characteristics. Feel free to replace these with whatever makes sense for you.



Sharpen your pencil

On page 60, you identified architectural characteristics for Sillycon Symposia's social media application, Lafter. To make sure you've achieved a good balance, limit your list to seven characteristics. Then, check the boxes next to the top three most important.

Top 3	Driving Characteristics	Implicit Characteristics
☐		*feasibility (cost/time)*
☐		*security*
☐		*maintainability*
☐		*observability*
☐		
☐		
☐		

These are implied characteristics. Move them to the Driving Characteristics column if you think they are critical to the success of the system.

Pick the top three most important ones (in any order).

Possible Candidate Architectural Characteristics

performance	data integrity	deployability
responsiveness	data consistency	testability
availability	adaptability	configurability
fault tolerance	extensibility	customizability
scalability	interoperability	recoverability
elasticity	concurrency	auditability

Solution on page 79

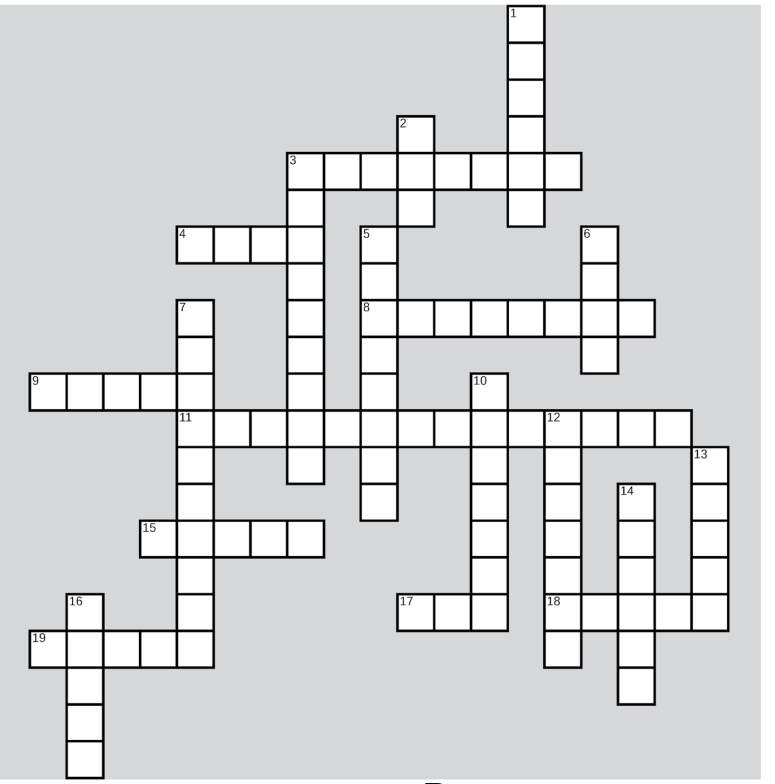
Bullet Points

- *Architectural characteristics* represent one part of the structural analysis that architects use to design software systems. (We'll talk abou the other part, logical components, in Chapter 4.)
- Architectural characteristics describe a system's capabilities.
- Some architectural characteristics overlap with operational concerns (such as availability, scalability, and so on).
- There are many catagories of architectural characteristics. No one can make a comprehensive list, because the software development ecosystem is constantly changing.
- When identifying architectural characteristics, architects look for factors that influence structural design.
- Architects should be careful not to specify too many architectural characteristics, because they are *synergistic*—changing one requires other parts of the system to change.
- Some architectural characteristics are *implicit*: not explicitly stated in requirements, yet part of an architect's design considerations.
- Some architectural characteristics may appear in multiple categories.
- Many architectural characteristics are *cross-cutting*: they interact with other parts of (and decisions in) the organization.
- Architects must derive many architectural characteristics from requirements and other domain design considerations.
- Some architectural characteristics come from *domain and/or environmental knowledge*, outside of the requirements of a specific application.
- Some architectural characteristics are *composites*: they consist of a combination of other architectural characteristics.
- Architects must learn to translate "business speak" into architectural characteristics.
- Architects should limit the number of architectural characteristics they consider to some small number, such as seven.



Characteristics Crossword

Ready to have some fun and test your knowledge about what you've learned? Try this crossword puzzle about architectural characteristics.



Across

3. An architectural characteristic that might be implicit
4. _____ engineering is an architectural problem
8. Choosing architectural characteristics means assigning each one a _____
9. Magic number of characteristics to ask for
11. A nonfunctional requirement is also called an architectural _____
15. Some architectural characteristics are _____-cutting
17. What architects should ask when users suggest solutions instead of requirements
18. A system might need to _____ up and down to meet demand
19. _____ tolerance is an architectural demand

Down

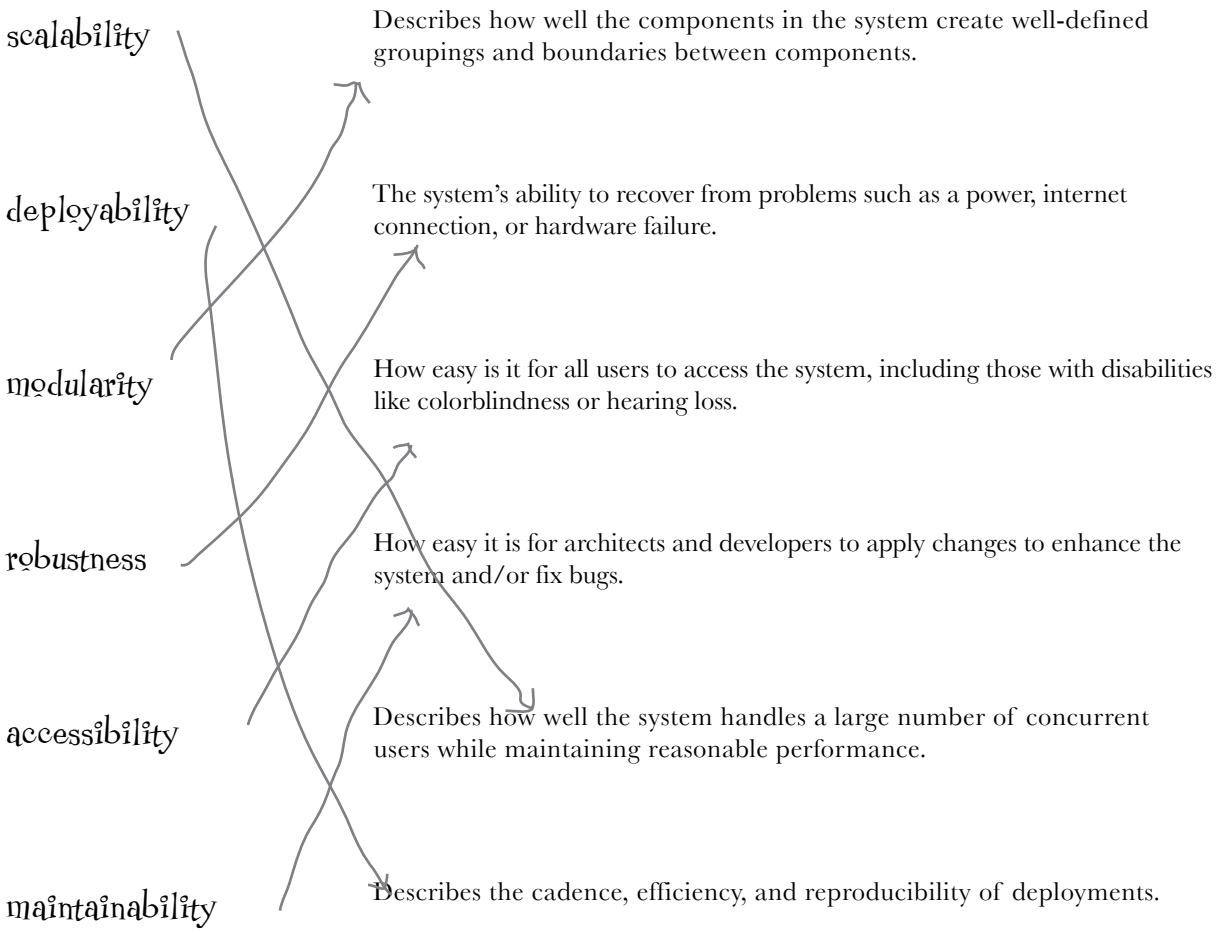
1. Uneven traffic often comes in _____
2. A Lafter post can be a "joke" or a "_____"
3. Architectural characteristics influence the system's _____
5. Architectural characteristics can be explicit or _____
6. _____ integrity is an architectural characteristic
7. A site with large numbers of _____ users might need to be scalable
10. Many governments regulate data _____
12. An architectural characteristic is critical or important to the system's _____
13. Combining architectural characteristics and logical components gives you an architectural _____
14. The thing you're writing software about
16. Web page loading time is often called "first contentful _____"

Solution on page 80

From page 56

Who Does What? Solution

So many architectural characteristics! We had a nice database that listed the ones that are most important for Lafter, along with their definitions, but somehow the index became corrupted and we lost the linkage. Can you help restore them by drawing a line from each architectural characteristic to its definition?



From page 58



Exercise Solution

Welcome to "Take It or Leave It!" The rules of this game are simple—we'll give you a business requirement that might come up for the Lafter application, and two architectural characteristics. As you know, everything in architecture is a trade-off, so if you attempt to optimize one, you probably won't do as well with the other. Your job is to tell us which characteristic you rate as a higher priority for that requirement.

"We need to get this to market ASAP!"	fault tolerance	agility
"Money's tight, folks!"	scalability	simplicity
"Oh, wow, this conference is going to be our biggest yet."	high availability	maintainability
"We want to start storing users' credit card information."	security	recoverability
"This site is going to be very popular upon launch."	agility	elasticity

From page 60



Exercise Solution

Domain requirements are often a rich source of architectural characteristics. For example, our Lafter application needs to support large numbers of users, so scalability will be one necessary characteristic. Can you uncover more? Here's what we came up with.

Users: hundreds of speakers, thousands of users

Requirements:

- Users can register for usernames and approve the privacy policy
- Users can add new content on Lafter as a "Joke" (long-form post) or "Pun" (short-form post)
- Followers can "HaHa" (indicating strong approval) or "Giggle" (a milder approval message) content they like
- Speakers at Sillycon Symposia events have a special icon
- Speakers can host forums on the platform related to their content
- Users can post messages of up to 281 characters
- Users can also post links to external content

Additional Context:

- International support
- Very small support staff
- "Bursty" traffic: extremely busy during live conferences

Architectural characteristics

- Scalability
- Elasticity
- Authorization
- Authentication
- Internationalization
- Customizability

From page 62



Exercise Solution

It's sometimes difficult to distinguish requirements from solutions. Here are some responses you might get when you ask "why?" that indicate something might be one or the other. Can you identify which indicate requirements and which solutions?

"We need a system to track user preferences and customizations, then save them between sessions."

☒ **requirement** ☐ **solution**

"Do we really need to build our own survey service? Surely we can find one that does what we need."

☒ **requirement** ☐ **solution**

"An enterprise service bus would solve some of our current problems (albeit with some changes and workarounds) and it offers extreme extensibility."

☐ **requirement** ☒ **solution**

"According to the friendly sales rep, this software package does all the things accounting needs, now and in the near future."

☐ **requirement** ☒ **solution**



Exercise Solution

From page 64

Context matters. At the top, we've listed several architectural characteristics. Below that are three application scenarios. For each scenario, rank each characteristic based on how important it is for that type of application. *Hint: Some applications won't need all of them.* Here are our rankings.

Scalability

Performance

Extensibility

Security

Data integrity

Scenario #1

An ecommerce site in a competitive market

1. Security
2. Performance
3. Scalability
4.
5.

Scenario #2

A system for an enterprise whose goal is to grow via mergers

1. Extensibility
2. Scalability
3.
4.
5.

Scenario #3

An application to automate standardized testing and grading for university admissions

1. Data integrity
2. Security
3. Performance
4.
5.



Exercise Solution

From page 71

On page 60, you identified architectural characteristics for the Sillycon Symposia Lafter social media application. To make sure you have achieved a good balance, limit the number to seven. Then, check the boxes next to the top three most important.

Top 3	Driving Characteristics	Implicit Characteristics
☒	Scalability	*feasibility (cost/time)*
☒	Security	*security*
☐	Elasticity	*maintainability*
☒	Responsiveness	*observability*
☐	Performance	
☐	Portability	
☐	Accessibility	

Remember, no single correct answer exists. The question is: can you justify your choices?

Possible Candidate Architectural Characteristics

performance	data integrity	deployability
responsiveness	data consistency	testability
availability	adaptability	configurability
fault tolerance	extensibility	customizability
scalability	interoperability	recoverability
elasticity	concurrency	auditability

Characteristics Crossword Solution

From page 73

